

Transition: DS + Algorithms

Vectors +
Lists



First: Worksheet

$$T(n) = aT\left(\frac{n}{b}\right) + n^k$$

$$a) A(n) = 4A\left(\frac{n}{3}\right) + n^2$$

$a=4$ $b=3$ $k=2$

$$a \text{ vs } b^k$$
$$4 < 3^2 \Rightarrow O(n^2)$$

$$b) B(n) = 3B\left(\frac{n}{5/3}\right) + n^1 = O(n^{\log_{5/3} 3})$$

$$\frac{3n}{5} = \frac{3}{5} \cdot n = \frac{n}{5/3}$$

$$a=3$$
$$b=5/3$$
$$k=1$$

$$a \text{ vs } b^k$$
$$3 > (5/3)^1$$

$$\begin{aligned}
 & \sum_{i=0}^{\log_{5/3} n} 3^i \cdot \left(\frac{3}{5}\right)^i n = n \sum_{i=0}^{\log_{5/3} n} \left(\frac{9}{5}\right)^i \\
 & = n \left(\left(\frac{9}{5}\right)^{\log_{5/3} n + 1} - 1 \right) \\
 & \quad \swarrow \left(\frac{9}{5} - 1 \right) \\
 & = n \cdot \frac{5}{4} \left(\left(\frac{3^2}{5}\right)^{\log_{5/3} n + 1} - 1 \right) \\
 & \quad \swarrow \left(\frac{9}{5} \right) \cdot \left(\frac{9}{5}\right)^{\log_{5/3} n} \\
 & = n \cdot \frac{5}{4} \cdot \frac{9}{5} \left(\left(\frac{9}{5}\right)^{\log_{5/3} n} - 1 \right) \\
 & \quad \swarrow \left(\frac{9}{5} \right)^{\log_{5/3} n} = \frac{3^2}{5} \\
 & = n \cdot \frac{5}{4} \cdot \frac{9}{5} \left(\frac{3^2}{5} - 1 \right) \\
 & = n \cdot \frac{5}{4} \cdot \frac{9}{5} \cdot \frac{4}{5} \\
 & = n \cdot \frac{9}{5}
 \end{aligned}$$

$\log_{5/3} n$
 $\sum_{i=0}^{\log_{5/3} n} 3^i \cdot \left(\frac{3}{5}\right)^i n$
 $\frac{3^i \cdot n}{5^i} = \frac{n}{(5/3)^i}$
 $\log_{5/3} n$
 a
 $\log_a a$
 $k \cdot n = \frac{n}{1/k}$
 $\left(\frac{3^2}{5}\right)^{\log_{5/3} n + 1}$
 $\left(\frac{9}{5}\right)^{\log_{5/3} n}$
 $\frac{9}{5}$
 $\frac{3^2}{5}$

$$= n^1 n^{\log_{5/3} \frac{9}{5}}$$

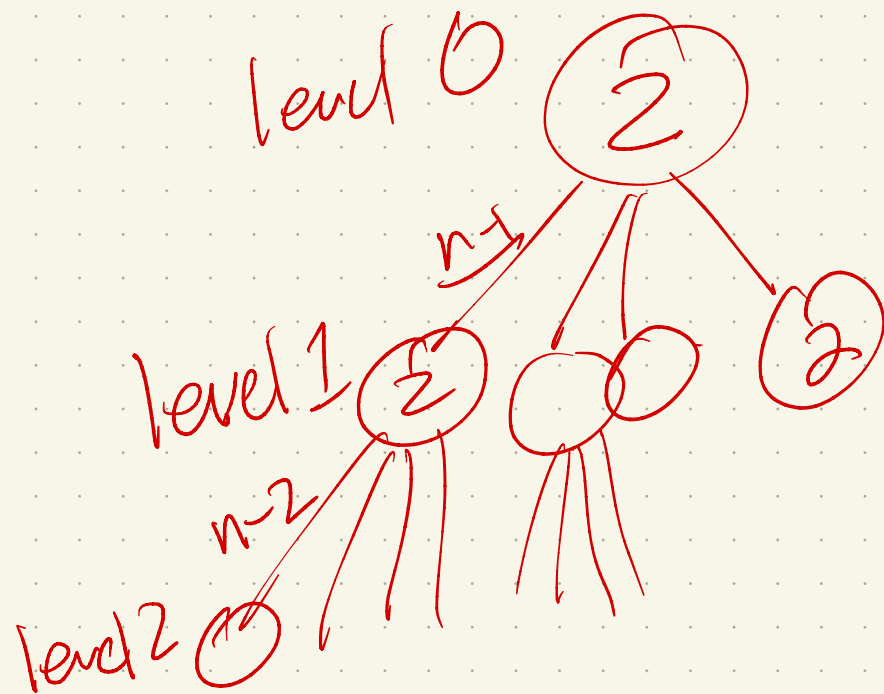
$$= n^{1 + \log_{5/3} \frac{9}{5}}$$

$$= n^{\log_{5/3} (5/3) + \log_{5/3} (9/5)}$$

$$= n^{\log_{5/3} \left(\frac{5}{3} \cdot \frac{9}{5} \right)}$$

$$= n^{\log_{5/3} 3}$$

c) $C(n) = 4C(n-1) + 2$ $x-4=0$
 $\hookrightarrow$ linear inhom. (poly deg 0)ⁿ



level
d=n
2

$$= \sum_{i=0}^n 4^i \cdot 2$$

$$= 2 \cdot \sum_{i=0}^n 4^i$$

$$= 2 \cdot \frac{4^{n+1} - 4}{4 - 1}$$

level i :
 4^i nodes $= O(4^n)$

Linked Lists vs. Arrays or Vector

often by restricted language

Both store "a list".

Operations:

- begin
- end
- empty
- front
- back
- insert
- delete

So: what's the difference?

- access position

- size

;

Arrays or Vectors:

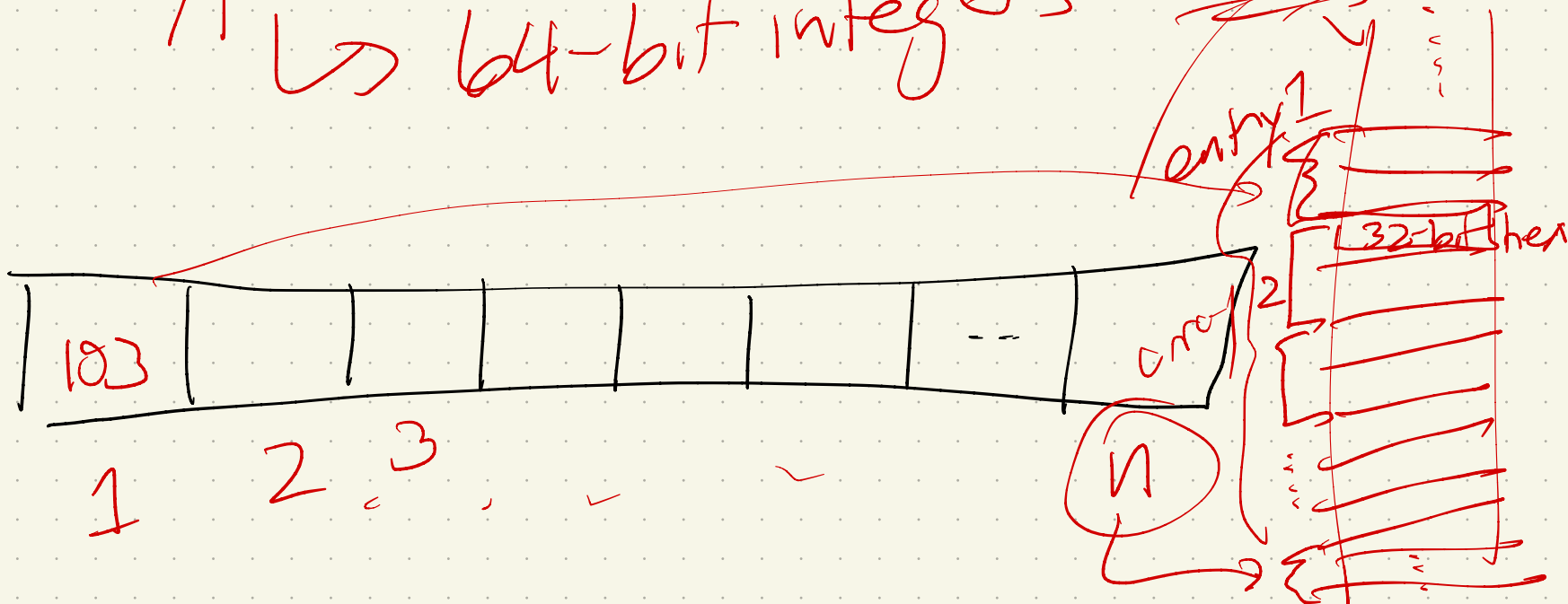
Assume data is stored in a contiguous region:

Need to know:

- length of array
- type of data
 - ↳ 64-bit integers

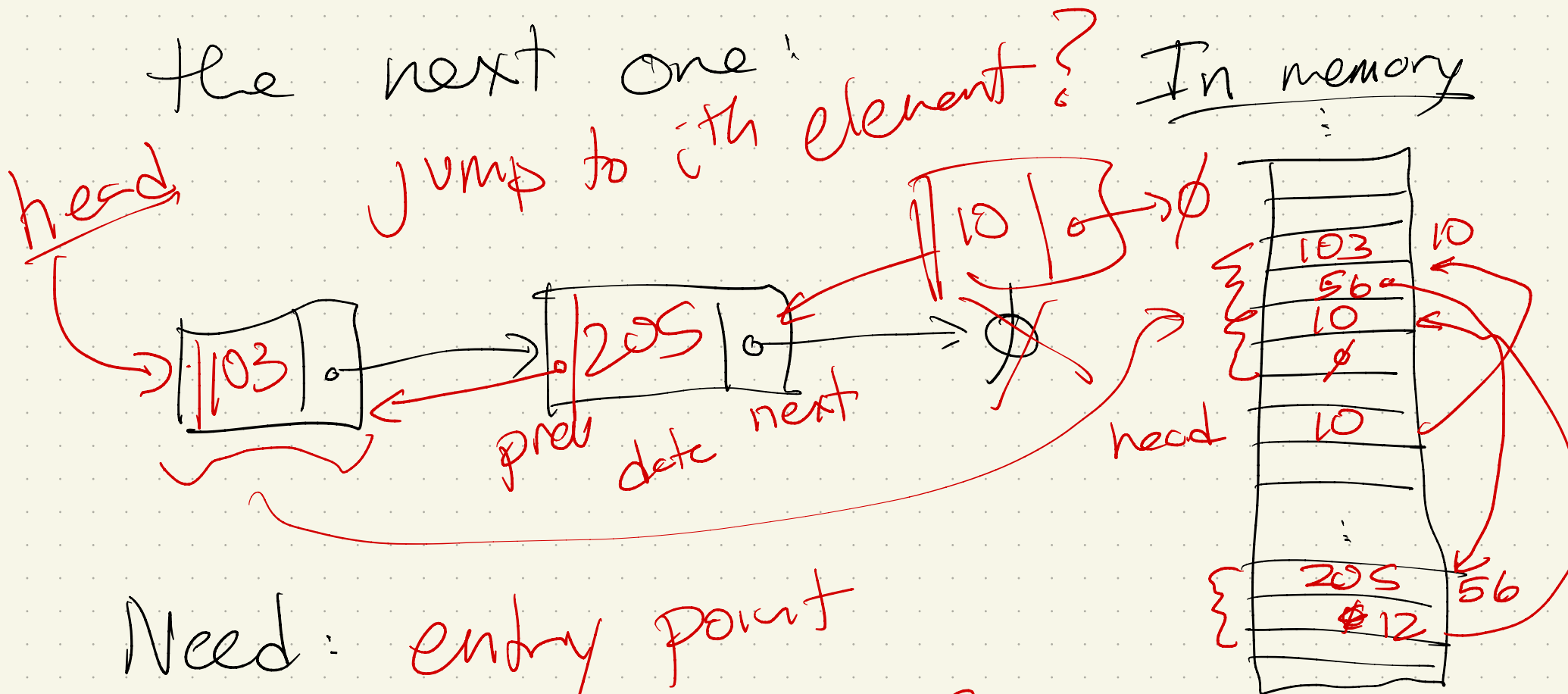
advantage
↳ access
elt at
pos $i \leq n$

$O(1)$



Linked lists:

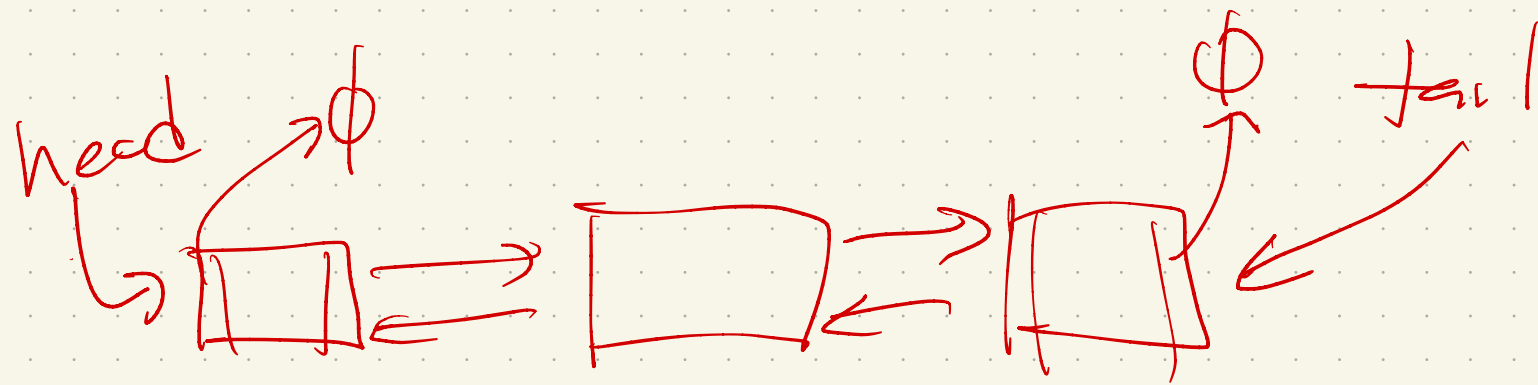
Not contiguous $\rightarrow$ instead, each element stores address of the next one.



Need: entry point

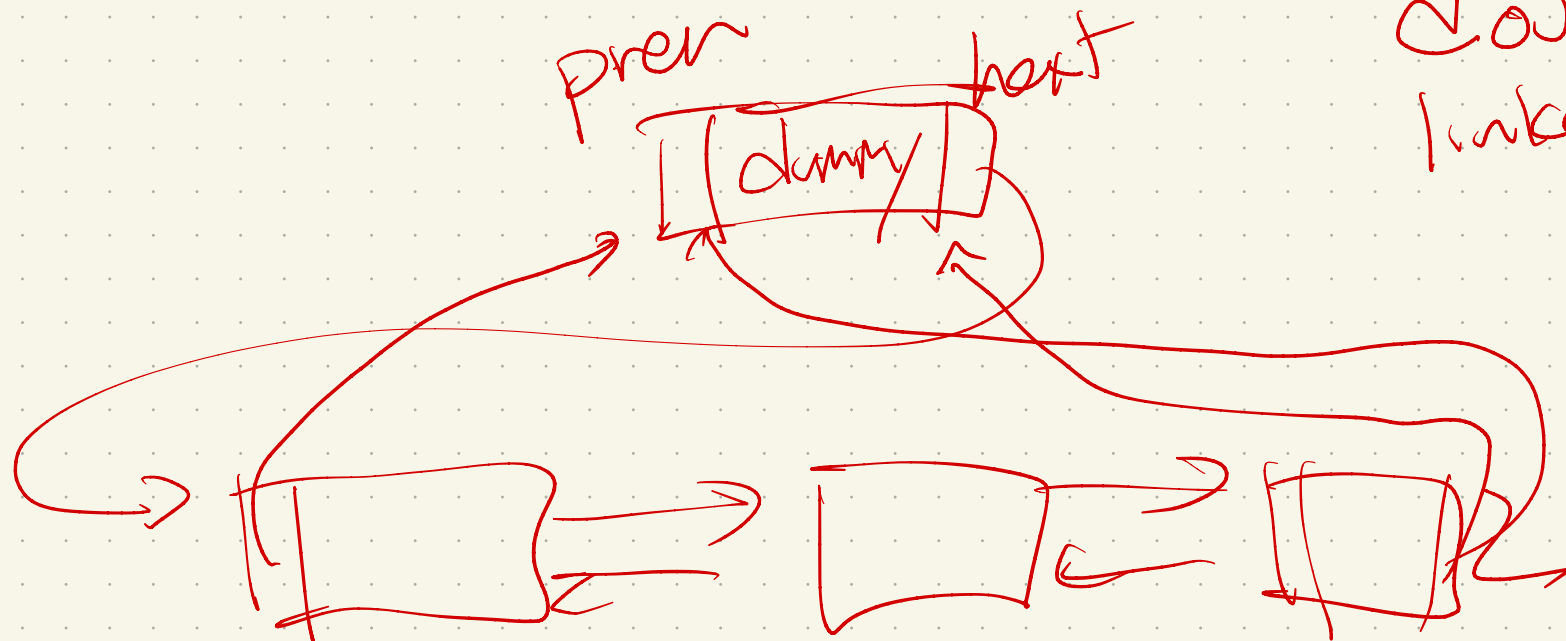
advantage: no max size

• no fixed type



Other implementations:

Circularly
double
linked list

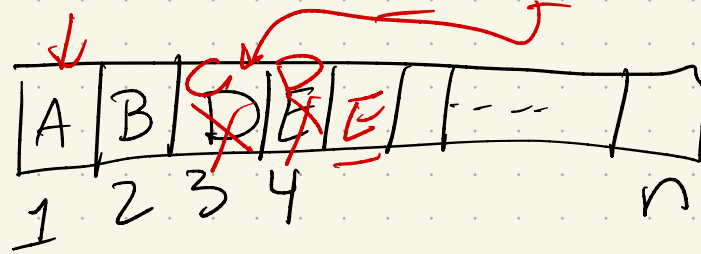


Trade-offs: array

Consider

Vector

insert(C, 3):



$O(\text{size})$

↓
make room!

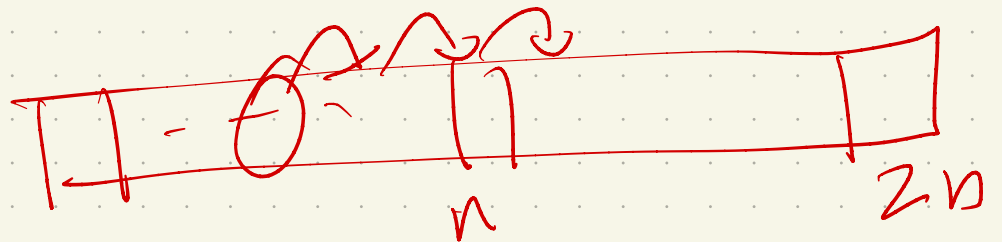
loop to
move $i+1$ to size down

max cap = n
size = 4

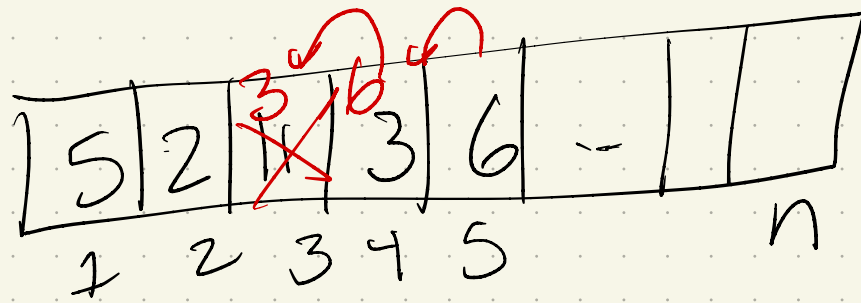
What if full?

size = n

↳ resize



Similar issue with delete:

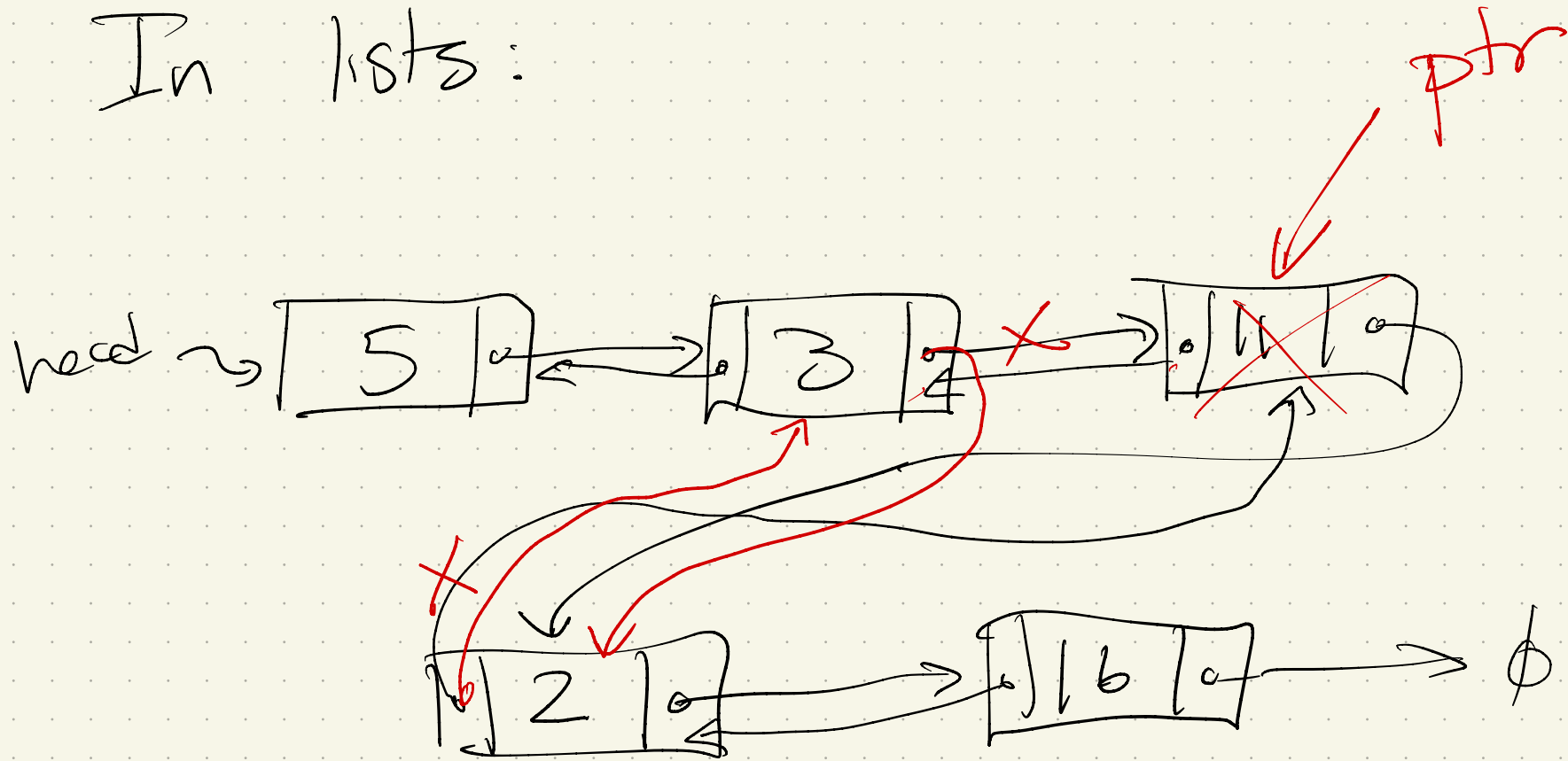


delete(3)

$O(n)$

to move things
down so no holes.

In lists:



delete 3: if I give you
"direct pointer" $O(1)$ →
just reset a few pointers

In lists, recursion comes up

naturally:

current ~~start~~ = head

access(i , current):

if $i = 1$

return data in current

else

return ($i-1$, current $\rightarrow$ next)

head

3

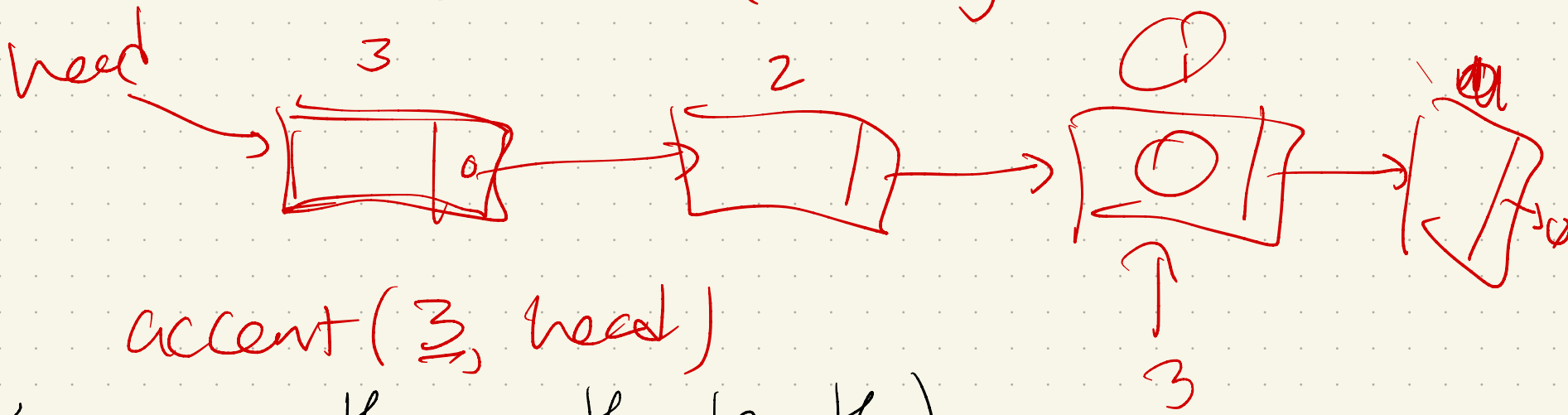
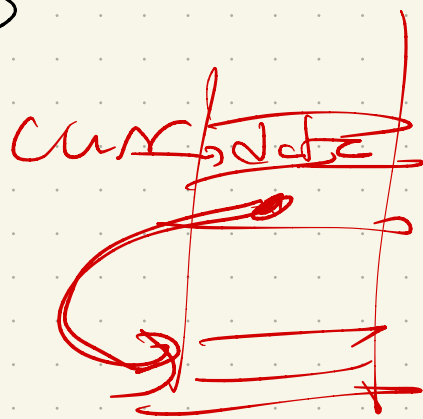
2

1

0

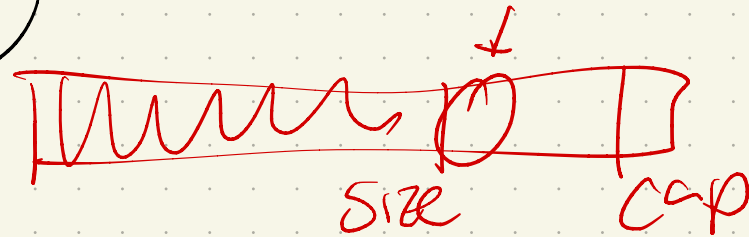
access(3, head)

(You saw this with length.)



Some special cases:

Inserting at the end of a vector:



unless full

$O(1)$ time

Inserting or deleting in a list
given a pointer to the node:

$O(1)$

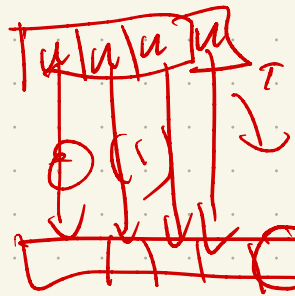
if I don't have ptr: find
element $O(n)$

Run times

	List	Vector
insert	if give ptr: $O(1)$	$O(n)$ (to move space)
insert at end	$O(1)$	$O(1)$ as long as not full $O(n)$ if full
access position i	$O(n)$	$O(1)$
delete	$O(1)$ if give ptr	$O(n)$

Amortized analysis

If we insert at end of a vector
n times:



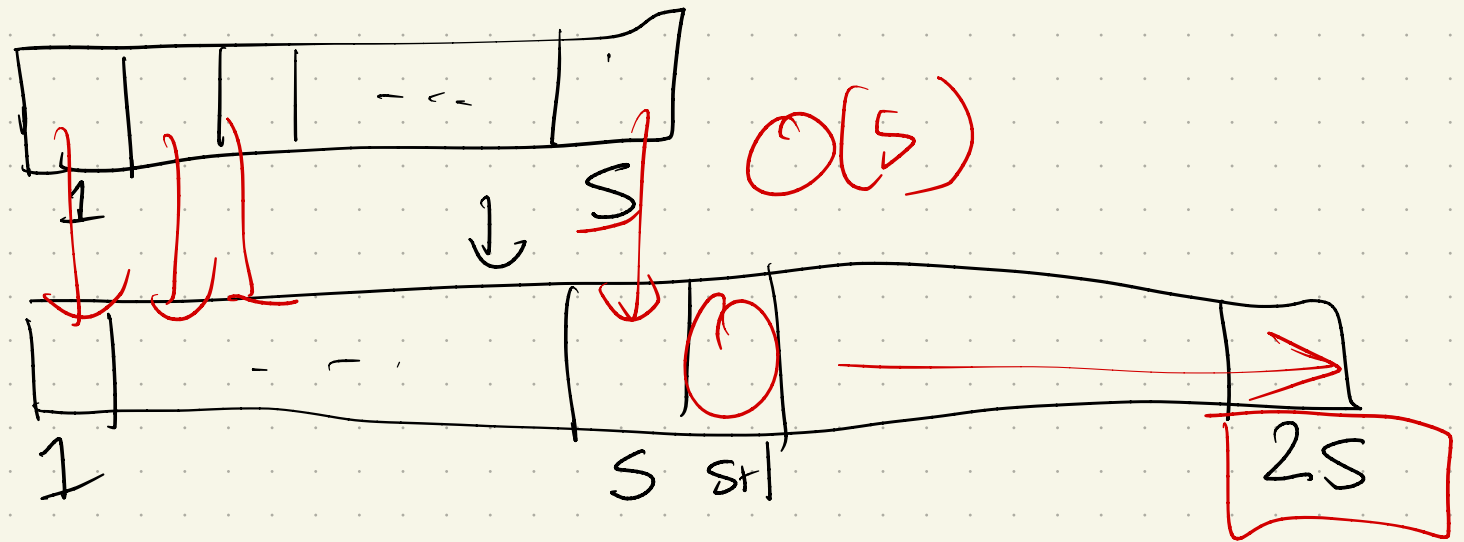
$O(1)$ ←

Except: When "full", can make larger
array & copy down, so we have
room.

Time: $O(n)$

So: what is time for n insertions
at end of a vector?

Note: Every time we double array
size, we have more room!



Formalize: what is average cost, if we
do many?

Claim: If we start with an empty vector, total time to do n insertions at end is $O(n)$.
assuming double vector when full

proof:

use "accounting" method:

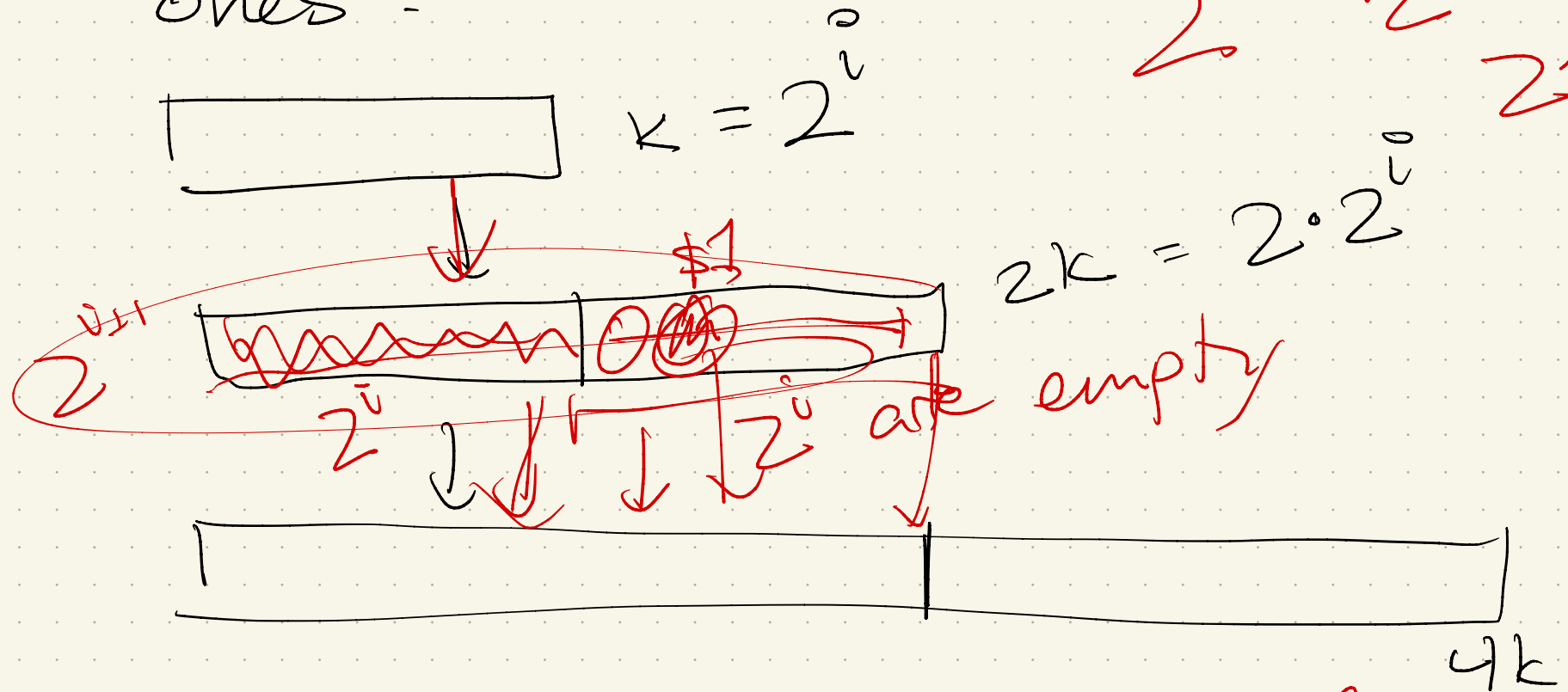
Each $O(1)$ operation costs \$1.

So non-overflow insert: \$1

& overflow insert: \$n

Then, overcharge the non-overflow ones:

$$2^i \cdot \$2 = 2 \cdot 2^i$$



2^i inserts until next full
 → then pay $2 \cdot 2^i$
 charge each insert \$3

Implications:

Which to use?

Depends!